Helix Constitution — Universal AGENTS.md

Field	Value
Revision	4
Created	2026-05-14
Last modified	2026-05-20
Status	active
Status summary	Mirrored Constitution.md §11.4.76 (containers-submodule mandate) into this AGENTS.md. §11.4.73 + §11.4.74 + §11.4.75 mirrors continue from earlier Revisions.
Issues	none
Issues summary	—
Fixed	§11.4.73 + §11.4.74 + §11.4.75 + §11.4.76 mirrors
Fixed summary	§11.4.76 lands in lockstep with the Constitution.md addition + QWEN.md creation.
Continuation	—

Table of contents

- Universal-vs-project classification (§11.4.17, User mandate 2026-05-14)
- Subagent-driven-by-default (§11.4.20, User mandate 2026-05-14)
- Script documentation mandate (§11.4.18, User mandate 2026-05-14)
- Fixed-document column-alignment (§11.4.19, User mandate 2026-05-14)
- Operator-blocked status + self-resolution exhaustion (§11.4.21, User mandate 2026-05-14)
- Document-sync commit discipline (§11.4.22, User mandate 2026-05-14)
- Build-resource stats tracking (§11.4.24, User mandate 2026-05-14)
- Full-Automation-Coverage (§11.4.25, User mandate 2026-05-15)
- Constitution-Submodule Update Workflow (§11.4.26, User mandate 2026-05-15)
- Submodule-dependency-manifest (§11.4.31, User mandate 2026-05-15)
- Post-constitution-pull validation (§11.4.32, User mandate 2026-05-15)
- .gitignore + no-versioned-build-artifacts (§11.4.30, User mandate 2026-05-15)
- Lowercase-snake_case-naming (§11.4.29, User mandate 2026-05-15)
- Submodules-as-equal-codebase + decoupling + dependency-layout (§11.4.28, User mandate 2026-05-15)
- No-fakes-beyond-unit-tests + 100%-test-type-coverage (§11.4.27, User mandate 2026-05-15)
- Type-aware closure-status vocabulary (§11.4.33, User mandate 2026-05-15)
- Reopened-source attribution (§11.4.34, User mandate 2026-05-15)
- Canonical-root inheritance clarity (§11.4.35, User mandate 2026-05-15)
- Mandatory install_upstreams on clone/add (§11.4.36, User mandate 2026-05-15)
- Fetch-before-edit (§11.4.37, User mandate 2026-05-15)
- Installable-asset evidence (§11.4.38, User mandate 2026-05-17)
- Full-suite retest before release tag (§11.4.40, User mandate 2026-05-17)
- Pre-force-push merge-first (§11.4.41, User mandate 2026-05-17)
- Iteration-discipline mandate (§11.4.42, User mandate 2026-05-18)
- TDD-Fix-Discipline (§11.4.43, User mandate 2026-05-18)
- Document revision header (§11.4.44, User mandate 2026-05-18)
- Integration-status-doc maintenance (§11.4.45, User mandate 2026-05-18)

    - Validate-recent-work-before-post-flash-tests (§11.4.46, User mandate 2026-05-18)
    - Firebase data review (§11.4.47, User mandate 2026-05-18)
    - UI-driven video testing (§11.4.48, User mandate 2026-05-18)
    - Dual-approach testing (§11.4.49, User mandate 2026-05-18)
    - Deterministic consistency (§11.4.50, User mandate 2026-05-18)
    - Live-ADB-First maximization (§11.4.51, User mandate 2026-05-18)
    - Autonomous-Validation (§11.4.52, User mandate 2026-05-18)
    - Fixed_Summary parity (§11.4.53, User mandate 2026-05-18)
    - ATM-NNN ticket identifier (§11.4.54, User mandate 2026-05-19)
    - Reopens-history tracking + per-item Reopens.md (§11.4.55, User mandate 2026-05-19)
    - Status_Summary parity + two-audience format (§11.4.56, User mandate 2026-05-19)
    - README.md doc-link section + revision metadata (§11.4.57, User mandate 2026-05-19)
    - Parallel-development methodology (§11.4.58, User mandate 2026-05-19)
    - README always-sync (§11.4.59, User mandate 2026-05-19)
    - Documentation always-sync composite covenant (§11.4.60, User mandate 2026-05-19)
    - Credentials-handling (§11.4.10)
    - Host-session safety (§12)
    - Continuation document (§12.10)
    - Data safety (§9)
- CLI workflow expectations
- Hierarchy of project authority
- When stuck

> This is the **base AGENTS.md** for any CLI agent (Codex, Cursor, Aider, Continue, Gemini CLI, future LLMs) working on a project that includes the Helix Constitution submodule. Project-level `AGENTS.md` may extend or tighten rules but MUST NOT weaken them.
>
> Last revision: 2026-05-14

How inheritance works

Because not every CLI agent honours the Markdown `@import` syntax, each consuming project's root `AGENTS.md` MUST do one of:

1. **Reference + restate critical rules** (safest — works for every agent):

```
# <Project> — AGENTS.md

> Base agent rules: `constitution/AGENTS.md` — READ IT FIRST.
> The base file is authoritative for any topic not covered
        here.

## Critical base rules restated (for agents that don't follow
        links)
- Never commit secrets.
- All commits go through the project's commit wrapper.
- Anti-bluff covenant binds every test (Constitution §11.4).

## Project-specific
- ...
```

2. **Generate at build time** — keep one source-of-truth in this submodule, concatenate with a project-specific delta file. A `compose-agents.sh` script in this submodule's `Upstreams/` (or `scripts/` if added) would output `constitution/AGENTS.md` followed by `---` followed by `AGENTS.project.md`.

Identity & posture

You are working inside a project that has opted into the Helix Constitution. The Constitution is the source of truth for engineering discipline (test coverage, anti-bluff covenant, data safety, host safety, credentials handling, documentation discipline). Whenever this AGENTS.md mentions a `§X.Y` reference, it points to `constitution/Constitution.md`.

The Constitution applies recursively to every submodule of every project that includes it — you MUST treat the Constitution's rules as in-force regardless of how deep into the submodule tree you are working.

Top-level invariants for every agent

1. **Never assume; verify by reading files.** Especially before any destructive action.
2. **Never bluff.** Do not invent file paths, command outputs, test results, or library APIs you have not verified. If you are unsure, say so explicitly and verify.
3. **No guessing language** (§11.4.6). `likely`, `probably`, `maybe`, `might`, `appears`, `seems` are forbidden when reporting causes. Use captured evidence or mark explicitly `UNCONFIRMED:`.
4. **Test coverage for every change** (§1) — pre-build gate, post-build gate, runtime test, AND a meta-test mutation that proves the gate catches regressions.

5. **Commit through the project's official wrapper.** Direct `git add` / `git commit` / `git push` on the main repo is forbidden in normal workflow.
6. **Never skip hooks** (`--no-verify`, `--no-gpg-sign`).
7. **Never force-push.** Force-push requires explicit per-session human authorization AND a green §9.1.5 post-op gate.
8. **Hardlinked backup before any destructive op** (§9). Zero excuse.
9. **Host safety** (§12) — abort if pre-flight check fails; wrap heavy work in bounded execution scopes; never exceed 60% of host RAM.
10. **Credentials NEVER tracked** (§11.4.10) — `.env` patterns git-ignored; runtime-load only; per-service file separation. **Pre-store leak audit** (§11.4.10.A, 2026-05-17) — before storing operator-provided credentials in any gitignored config, grep the entire tracked tree AND git history for the literal value(s); surface any findings to operator before storing.
11. **CONTINUATION.md kept in sync** (§12.10) — every non-trivial state change updates the continuation document in the same commit.

Critical base rules restated (for agents that don't honour @imports)

Anti-bluff covenant — END-USER QUALITY GUARANTEE (§11.4)

Forensic anchor — verbatim user mandate (2026-04-28):

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

The bar for shipping is **not** "tests pass" but **"users can use the feature."** Every PASS MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, absence-of-error PASS, and grep-based PASS without runtime evidence are critical defects.

Mutation-paired gates (§1.1)

Every new gate MUST be paired with a mutation entry in the project's meta-test harness that mutates the asserted condition and asserts the gate now FAILs. A gate without a paired mutation is a bluff gate and is forbidden.

Recorded-evidence requirement (§11.4.2)

Every PASS for a user-visible feature MUST be cross-checked against a captured recording + action timeline. A PASS that lacks at least one matched timeline event in the analyzer findings is a §11.4 PASS-bluff.

Test-interrupt-on-discovery (§11.4.4)

The moment any defect is re-discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop. Then: systematic debugging → fix at root cause → four-layer test coverage (pre-build / post-build / runtime / meta-test paired mutation) → full rebuild → re-deploy on every target → full retest from beginning.

No-guessing mandate (§11.4.6)

Forensic anchor — verbatim user mandate (2026-05-08):

> "'LIKELY' is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainity is allowed at any cost! We MUST always know exactly precisly what is happening exactly, in any context, under any conditions, everywhere!"

Forbidden vocabulary when reporting causes: `likely`, `probably`, `maybe`, `might`, `possibly`, `presumably`, `seems`, `appears to`, `guess`, `seemingly`, `apparently`, `perhaps`, `supposedly`, `conjectured`, and synonyms.

Item-type tracking (§11.4.16)

Every active item in the project Issues file MUST carry a `**Type:**` line within eight non-blank lines of its heading. Three-value CLOSED vocabulary: `Bug` (product defect / regression / user-visible broken behaviour), `Feature` (new capability not previously offered to end users), `Task` (internal workstream — refactor, doc, infra, gate, audit; the lowest-stakes default when ambiguous). The Issues_Summary file carries the Type column for every active item. All three Issues / Issues_Summary / Fixed file types kept in

sync (Markdown + HTML + PDF). Pre-build gates `CM-ITEM-TYPE-TRACKING` + `CM-COVENANT-114-16-PROPAGATION` enforce the mandate. No escape hatch.

Universal-vs-project classification (§11.4.17, User mandate 2026-05-14)

Before adding ANY new rule / mandatory constraint / covenant clause / gate / "MUST"-bearing statement, classify it: **universal** (any project → constitution submodule) or **project-specific** (specific hardware / vendor / package / region → project / submodule layer). Commit message MUST carry `Classification:` line + one-sentence rationale. When uncertain, default to project-specific. Gate `CM-UNIVERSAL-VS-PROJECT-CLASSIFICATION` audits new rule commits. Paired mutation enforces the gate is not a bluff.

Subagent-driven-by-default (§11.4.20, User mandate 2026-05-14)

When the runtime supports subagent delegation (Agent tool / task runner / sub-session), the primary agent MUST default to subagent delegation for multi-step scope (≥3 phases), parallelisable independent subtasks, long-running diagnostic loops, OR specialised domain workflows. Foreground-only is reserved for single-file edits, mid-execution operator clarification, critical-state sequencing (commits / pushes / tags), or tasks under ~30s. Discipline: tight scope (4-6 tasks), checkpoint commits per task, anti-stall protection in prompts, anti-bluff verification of subagent claims via repo state. Parallel subagents MUST partition non-overlapping files; parent `commit_all.sh --auto-cascade` bundles via `git add -A`. Gate `CM-SUBAGENT-DELEGATION-AUDIT` (per consuming project) flags foreground multi-step work as a violation. Paired mutation enforces the gate is not a bluff.

Script documentation mandate (§11.4.18, User mandate 2026-05-14)

Every Bash / shell / POSIX-sh script under `scripts/` / `bin/` / `tests/` / library dirs / CI hooks (depth-N recursive) MUST carry (1) an in-source documentation block at the top (Purpose / Usage / Inputs / Outputs / Side-effects / Dependencies / Cross-references) AND (2) an external user guide under `docs/scripts/<script-name>.md` (Overview / Prerequisites / Usage examples / Edge cases / Internal behaviour / Related scripts / Last verified date). When a script is modified, BOTH the in-source block AND the external user guide MUST be updated in the SAME commit. **No documentation ever out of sync with its codebase.** Gate `CM-SCRIPT-DOCS-SYNC` enforces. Paired mutation strips the invariant.

Fixed-document column-alignment (§11.4.19, User mandate 2026-05-14)

The closed-archive tracker (`Fixed.md`) MUST mirror the open-work tracker (`Issues.md` + `Issues_Summary.md`) along the same lifecycle axes — at minimum **Status** and **Type**. Concretely: (1) every `###` / `####` heading in `Fixed.md` carries `**Status:**` (from closed-set `{Fixed (→ Fixed.md) | Fixed — pending device verification | Fixed — RECLASSIFIED}`) and `**Type:**` (`{Bug | Feature | Task}`) within 8 non-blank lines of the heading; (2) `Fixed_Summary.md` companion exists with column structure `# | Level | Status | Type | One-line description` matching `Issues_Summary.md` exactly; (3) all three formats (`.md`, `.html`, `.pdf`) for BOTH Fixed and Fixed_Summary stay in sync via the same single-shot wrapper that handles Issues + Issues_Summary; (4) closure is atomic — resolved items migrate from Issues.md to Fixed.md in the same commit. Pre-build gate `CM-FIXED-COLUMN-ALIGNMENT` (5+ invariants: file-exists, header-has-Status+Type, mtime ≥ Fixed.md, generator script present, sync wrapper invokes it, HTML+PDF exports present). Paired mutation strips Status column from Fixed_Summary header → gate FAILs. Classification: universal (per §11.4.17).

Operator-blocked status + self-resolution exhaustion (§11.4.21, User mandate 2026-05-14)

`Operator-blocked` is the §11.4.15 Status closed-set's 7th value alongside `{Queued | In progress | Ready for testing | In testing | Reopened | Fixed (→ Fixed.md)}`. **Last-resort classification** — agent MUST first exhaust applicable self-resolution paths: (a) CLI / ADB / SSH / API access already available, (b) subagent delegation per §11.4.20, (c) existing repo tooling, (d) captured fallback (synthetic event / asset substitution / §11.4.3 SKIP), (e) external research per §11.4.8. Every `Operator-blocked` item MUST carry `**Operator-Block-Details:**` within 8 non-blank lines of its heading stating WHAT (action) / WHY (each exhausted alternative) / UNBLOCK CONDITION (observable signal) / WHO (contact or doc pointer). `Issues_Summary.md` lists it as a sortable Status value. Items re-evaluated every Nth tag cycle (≥3rd recommended). Fake `Operator-blocked` without exhaustion audit = §11.4 covenant violation at planning layer (PASS-bluff equivalent). Gates `CM-ITEM-OPERATOR-BLOCKED-DETAILS` + `CM-OPERATOR-BLOCKED-SELF- RESOLUTION-AUDIT` (every NEW Operator-blocked commit carries an "Attempted: a — …; b — …; c — …" trail). Classification: universal (per §11.4.17). No escape hatch.

Document-sync commit discipline (§11.4.22, User mandate 2026-05-14)

Every project tracking work items through an Issues / Fixed lifecycle MUST provide a **lightweight commit path** distinct from the full-repo commit wrapper. Stages, commits, pushes ONLY the status-tracking doc set: Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + their HTML + PDF exports + auto-generated audit artifact. Wrapper MUST: (a) auto-invoke the project's export-regeneration pipeline first so Markdown + HTML + PDF stay in sync; (b) stage ONLY the explicit doc-set list — NEVER `git add -A`; (c) use a separate flock disjoint from the full-tree wrapper; (d) push to every parent-repo remote; (e) exit `3` on nothing-to-commit (informational). MUST be invocable standalone OR via a delegation flag on the full-tree wrapper (e.g. `commit_all.sh --docs-only`). Inherits §9 preflight. Gate `CM-COMMIT-DOCS-EXISTS` + paired mutation. Composes with §11.4.12 / §11.4.15 / §11.4.18 / §12.10. Classification: universal (per §11.4.17). No escape hatch — doc-status drift is a §11.4 PASS-bluff at the documentation layer.

Build-resource stats tracking (§11.4.24, User mandate 2026-05-14)

Every project under this Constitution with a build exceeding 1 minute wall-clock MUST run a host-side resource sampler for every build — samples /proc/meminfo + /proc/loadavg + /proc/stat + /proc/diskstats at a fixed interval (recommended 5 s); writes TSV. On stop, computes **min / max / mean / p95** per metric; appends one TSV row per build to a registry; regenerates a Markdown report (Stats.md) whose top surfaces **ever-values** (min / max / mean across all tracked builds) and whose body lists per-build entries most-recent-first with SUCCESS / FAIL / UNKNOWN + reason. Stats.md exported to Stats.html + Stats.pdf through the project's normal export pipeline per §11.4.12. Triple committed via the §11.4.22 lightweight doc-sync wrapper. Sampler MUST stay under 50 MB RSS / 5% CPU. Stop hook called from both success AND failure paths of the build wrapper. Gate `CM-BUILD-RESOURCE-STATS-TRACKER` + paired mutation. Classification: mixed (per §11.4.17). Composes with §11.4.12 / §11.4.18 / §11.4.22 / §12.6 / §12.7 / §12.9. No escape hatch — build-resource debugging without time-series data is the bluff this anchor forbids.

Full-Automation-Coverage (§11.4.25, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> “Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered!”

No feature / functionality / flow / use case / edge case / service / application on any supported platform may be considered deliverable until automation tests prove six invariants: (1) anti-bluff posture with captured runtime evidence (§7.1 + §11.4); (2) proof of working capability end-to-end on target topology (§11.4.3, no mocks); (3) implementation matches documented promise; (4) no open issues/bugs surfaced (cross-checked vs §11.4.15 / §11.4.16); (5) full documentation (user manual + §11.4.18 for scripts) kept in sync via §11.4.12; (6) four-layer test floor per §1 (pre-build + post-build + runtime + paired mutation). Consuming projects publish a coverage ledger (feature × platform × invariant × status) regenerated at release-gate sweep. Classification: universal (§11.4.17). Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. See Constitution §11.4.25 for the full mandate.

Constitution-Submodule Update Workflow (§11.4.26, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> “Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE commited and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!”

Before any modification to `constitution/{Constitution,CLAUDE,AGENTS}.md`, execute in order: (1) fetch + pull (–ff-only or –rebase; never strategy=ours / unrelated-histories without authorization) so the submodule is at upstream tip before editing; (2) apply the change with §11.4.17 classification + verbatim mandate quote; (3) validate (meta_test_inheritance.sh + no merge-conflict markers + cross-file consistency); (4) commit (governance files only, no `git add -A`) + push to EVERY configured upstream remote (§2.1 violation if not); (5) careful conflict resolution preserving union of governance content — force-push forbidden (§9.2); (6) post-merge validation: `git submodule update --remote --init` + re-run cascade verifier (CONST-047) confirming

the new clause reaches every owned submodule; (7) bump consuming project's `.gitmodules` pointer to new HEAD in the SAME commit as cascade work — out-of-sync pointer = §11.4.26 violation. Classification: universal (§11.4.17). Severity-equivalent to force-push without §9.2 authorization. No escape hatch. See Constitution §11.4.26 for the full pipeline (operational scope, cross-cutting reach).

Submodule-dependency-manifest (§11.4.31, User mandate 2026-05-15)

Every owned-by-us submodule MUST ship `helix-deps.yaml` listing its own-org dependencies: `{name, ssh_url, ref, why, layout: flat|grouped}`. Tooling `incorporate-submodule <ssh-url>` adds the submodule at the parent project's canonical path (CONST-051(C)) + reads its helix-deps.yaml + recurses + aborts on conflicting refs + emits `<root>/.helix-manifest.yaml` audit record. Anti-bluff: each manifest paired with a Challenge that bootstraps from scratch, asserts layout matches manifest, runs submodule tests, captures wire evidence. Without the proof, the manifest is a §11.4.31 violation. This rule is the operational complement of §11.4.28 / CONST-051(C) — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root. Classification: universal (§11.4.17). See Constitution §11.4.31 for the full mandate.

Post-constitution-pull validation (§11.4.32, User mandate 2026-05-15)

Whenever a project's constitution submodule is fetched + pulled with any content change, run `scripts/verify-all-constitution-rules.sh` BEFORE the new HEAD is treated as canonical for other work. The sweep re-runs the governance-cascade verifier + every implementable rule gate (CONST-053 .gitignore audit, CONST-051(C) nested-own-org audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke). Failures populate Issues per §11.4.15 (Status: Reopened, Type: Bug); closure requires positive-evidence per §11.4. Pull-time invocation auto-triggered by `git submodule update --remote constitution`. Anti-bluff: sweep's own meta-test (paired mutation §1.1) plants a violation per gate, asserts sweep FAILs. This is the **enforcement engine** for every other rule — without it, new rules are decorative anchors rather than enforced gates. Classification: universal (§11.4.17). See Constitution §11.4.32 for the full mandate.

.gitignore + no-versioned-build-artifacts (§11.4.30, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> “every project module, every Submodule, every servcie and apolication MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! … If any violetion is detected it MUST be fixed before commit is executed!”

Every project module / submodule / service / application MUST ship a proper `.gitignore`. Forbidden-from-version-control: build artefacts (`bin/`, `build/`, `dist/`, `target/`, `*.exe`, `*.so`, `*.class`, `*.pyc`), cache files (`__pycache__/`, `node_modules/`, `.gradle/`, `.terraform/`), temp files (`*.tmp`, `*.swp`, `.DS_Store`), sensitive data (`.env`, `.env.*` except `.env.example` placeholder, `*.pem`, `*.key`, `id_rsa*`, `.netrc`, `secrets/`, `api_keys.sh`), generated logs (`*.log`, `coverage.out`, `htmlcov/`), OS/IDE personal state (`.idea/`, `.history/`).

Anti-bluff invariant: ignore-line alone is not enough — no file matching forbidden patterns may be tracked. Pre-commit attention: every author inspects `git diff --staged` + `git status` BEFORE commit; violations abort the commit (un-stage, add to ignore, scrub if already-tracked). Gate `CM-GITIGNORE-PRECOMMIT-AUDIT` + paired mutation. Recreatable-content test: if a documented mechanism regenerates the file from sources, it’s a build derivative and MUST be ignored.

Secret-leak intersection (§11.4.10 / CONST-042 / §12.1): a `.env` leak is BOTH a §11.4.30 and a §11.4.10 violation; rotation + post-mortem required.

Classification: universal (§11.4.17). Severity-equivalent to §11.4 PASS-bluff at the repository-hygiene layer. Composes with §1, §2, §9.1, §11.4.10, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25, §11.4.26, §11.4.27, §11.4.28, §11.4.29, CONST-047. See Constitution §11.4.30 for the full mandate.

Lowercase-snake_case-naming (§11.4.29, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> “naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of ’_’ character (snake-case)! All existing Submodules and directories which are not

> following this rule MUST BE renamed! … NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!”

Every directory / submodule / file MUST use lowercase snake_case. Non-compliant names MUST be renamed; every reference (configs, docs, source imports, governance files) updated atomically (reference drift = §11.4.29 violation of equal severity). Exceptions (common-sense, technology-preserving): language- mandated case for Java/Kotlin/Android/Apple/C#/Swift inside the language root; vendor third-party submodules; build artefacts.

`Upstreams/` → `upstreams/` transition: constitution submodule’s `install_upstreams.sh` MUST support BOTH directory names during migration; lowercase wins when both present.

Project-Toolkit Upstreamable machinery fetched+pulled before any rename batch + itself complies. Lacking BOTH-dir support is a release blocker.

Test coverage of renames: regression test for reference resolution + full CONST-050(B) test-type matrix + anti-bluff wire-evidence.

Phased execution: brainstorming → phase plan → fine-grained tasks/subtasks → every change covered. §11.4.20 subagent delegation for cross-cutting rename sweeps.

Classification: universal (§11.4.17). No escape hatch beyond common-sense exceptions. Severity-equivalent to §11.4 PASS-bluff at the reference-integrity layer. Composes with §1, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25, §11.4.26, §11.4.27, §11.4.28, CONST-047. See Constitution §11.4.29 for the full mandate.

Submodules-as-equal-codebase + decoupling + dependency-layout (§11.4.28, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> “All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory — we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project’s codebase! We MUST work on that code as much as we do with main project’s codebase! … We MUST NEVER modify Submodules to bring into them any project specific context … All

> Submodule dependencies that are used by Submodule MUST BE acessed from the root of the project! We MUST NOT have nested Submodule dependencies."

Three invariants. **(A) Equal-codebase**: every owned-by-us submodule (orgs: vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory — discoverable via gh/glab) is an equal part of the consuming project's codebase. Same engineering attention: analysis, extension, tests, gap-fill, bug-fix, documentation. Coverage ledgers list each submodule as in-scope. **(B) Decoupling**: NEVER inject project-specific context INTO submodules; they remain project-not-aware, reusable, modular, testable. When a submodule needs parent info, use configuration injection. **(C) Dependency-layout**: every dependency consumed by an owned submodule lives at `<root>/<name>/` or `<root>/submodules/<name>/` of the parent project. Nested own-org submodule chains FORBIDDEN — add the dependency at the parent root; the submodule reaches it via documented import/SDK/runtime resolver. Third-party submodules exempt.

Gates: `CM-OWNED-SUBMODULE-EQUAL-ENGINEERING`, `CM-OWNED-SUBMODULE-DECOUPLING`, `CM-OWNED-SUBMODULE-LAYOUT`. Paired mutations for each. Classification: universal (§11.4.17). No escape hatch. Severity-equivalent to a §11.4 PASS-bluff at the codebase- completeness layer. Composes with §1, §3, §11.4.17, §11.4.20, §11.4.25, §11.4.26, §11.4.27, CONST-047. See Constitution §11.4.28 for the full mandate.

No-fakes-beyond-unit-tests + 100%-test-type-coverage (§11.4.27, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type."

Two invariants. **(A)** Mocks/stubs/fakes/placeholders/TODOs/FIXMEs/"for now" patterns PERMITTED only in unit-test sources; non-unit tests (integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA) MUST exercise the real, fully implemented system. Production code MUST NOT import mock paths. Gate `CM-NO-FAKES-BEYOND-UNIT-TESTS` + paired mutation. **(B)** Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX,

Challenges (vasic-digital/Challenges submodule fully incorporated), HelixQA (HelixDevelopment/HelixQA submodule fully incorporated, with full autonomous QA sessions executing every registered test bank).

Required dependency submodules (recursive per CONST-047): Challenges (`git@github.com:vasic-digital/Challenges.git`) + HelixQA (`git@github.com:HelixDevelopment/HelixQA.git`) + any other functionality submodules under vasic-digital/HelixDevelopment orgs the project depends on. Pointers bumped to upstream HEAD in same commit as cascade work (§11.4.26 step 7).

Classification: universal (§11.4.17). Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. Composes with §1, §7.1, §11.4.1–§11.4.26 (esp. §11.4.25 — strict per-type-of-test expansion). See Constitution §11.4.27 for full mandate.

Type-aware closure-status vocabulary (§11.4.33, User mandate 2026-05-15)

Every project that tracks work items by Type per §11.4.16 MUST close them with the Type-appropriate closure-status word: `Bug` → `Fixed (→ Fixed.md)`, `Feature` → `Implemented (→ Fixed.md)`, `Task` → `Completed (→ Fixed.md)`. The `(→ Fixed.md)` suffix is preserved across all three so existing migration tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working. Generators treat the three terminal values as semantically equivalent (all closed, positive evidence captured) but preserve the literal in emitted docs. Closing a `Feature` with `Fixed (→ Fixed.md)` or a `Task` with `Implemented (→ Fixed.md)` is a §11.4.33 violation. Pre-build gate `CM-CLOSURE-VOCAB-TYPE-AWARE`. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23. Classification: universal (per §11.4.17). No escape hatch.

Reopened-source attribution (§11.4.34, User mandate 2026-05-15)

Every Issues.md heading whose `**Status:**` is `Reopened` MUST carry a `**Reopened-Details:**` line within 8 non-blank lines of the heading, capturing four sub-facts: **By:** `AI` or `User`; **On:** ISO date; **Reason:** one of `{ test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }` or explicit free text; **Evidence:** path or short description of the captured artefact. Reopens without evidence are §11.4.6 / §11.4.7 violations: the reopen IS a demotion-from-Fixed change. Issues_Summary.md Status column MUST distinguish Reopened sub-states by source (e.g., `Reopened (AI: test-failed)` vs `Reopened (User: manual-testing)`). Pre-build

gate `CM-ITEM-REOPENED-DETAILS` mirrors §11.4.21 walk pattern. Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21. Classification: universal (per §11.4.17). No escape hatch.

Canonical-root inheritance clarity (§11.4.35, User mandate 2026-05-15)

The constitution submodule's three files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`) ARE the canonical root — also called the parent files. Universal rules per §11.4.17 live here.

The consuming project's repository-root files (`<project-root>/CLAUDE.md`, `<project-root>/AGENTS.md`, optionally `<project-root>/Constitution.md` or equivalent) are consumer extensions. They open with the inheritance pointer (either `@constitution/CLAUDE.md` import or `## INHERITED FROM constitution/CLAUDE.md` heading). Project-specific rules per §11.4.17 live here.

When in doubt: universal rule → constitution submodule; project-specific rule → consumer's file. Default consumer-side when uncertain. "Parent CLAUDE.md" / "root Constitution" → constitution submodule file at `constitution/<filename>`. "Project CLAUDE.md" / "this project's AGENTS.md" → consumer file at `<project-root>/<filename>`. Moving a rule between layers MUST be a visible commit — `git mv` + explicit "Lifted from to constitution per §11.4.35" / "Demoted from constitution to per §11.4.35" line in the message. Pre-build gate `CM-CANONICAL-ROOT-CLARITY`. Composes with §11.4.17. Classification: universal (per §11.4.17). No escape hatch.

Mandatory install_upstreams on clone/add (§11.4.36, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!"

Every clone / add of a Git repository under any consuming project MUST be followed by `install_upstreams` invocation from the repository's root IF its tree contains `upstreams/` (or legacy `Upstreams/` per §11.4.29 transition) populated with `*.sh` recipe files. The utility (installed on operator's `PATH` via `.bashrc`/ `.zshrc`,

implementation lives in this constitution submodule) reads recipe files + configures every declared upstream as a named git remote + fans out `origin` push URLs.

Skipping the invocation when `upstreams/` is present silently breaks §2.1 (multi-upstream push is the norm). Gate `CM-INSTALL-UPSTREAMS-ON-CLONE` + paired mutation. Automation: `incorporate-submodule` (§11.4.31) auto-invokes; manual invocation also supported. Pre-commit check: `git remote -v | grep -c push` reports expected upstream count.

Classification: universal (§11.4.17). Composes with §2, §2.1, §3, §9.2, §11.4.17, §11.4.20, §11.4.28, §11.4.29, §11.4.30, §11.4.31. See Constitution §11.4.36 for the full mandate.

Fetch-before-edit (§11.4.37, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

> "Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them!"

FIRST git-touching action of any session: `git fetch --all --prune && git log --oneline HEAD..@{u} && git submodule foreach --recursive 'git fetch --all --prune --quiet'`. If `HEAD..@{u}` is non-empty, integrate (ff-merge / rebase / surface per §11.4.4) BEFORE any local edit, scanner, or test. Skipping on "nothing could have changed" is a §11.4.6 (no-guessing) violation — remote state is not knowable without a fetch.

The originating 2026-05-15 incident: agent ran `git remote remove gitee` as a fresh task, but the work had been committed upstream by a parallel agent 25 minutes earlier — local config edit was a no-op echo of already-merged history. A 30-second fetch would have caught it.

Scope: consuming project root + every owned submodule recursively (§11.4.28) + the constitution submodule itself (§11.4.26 step 1) + any dependency cloned via `incorporate-submodule` (§11.4.31) or `git submodule add` (§11.4.36). Gate `CM-FETCH-BEFORE-EDIT-AUDIT` + paired mutation per §1.1. Classification: universal (per §11.4.17). No escape hatch.

Installable-asset evidence (§11.4.38, User mandate 2026-05-17)

For any user-distributable build artifact (package, bundle, installer, or container image), tests/challenges MUST open the artifact and verify each user-visible asset is **present** and **non-degenerate**. A PASS without artifact-opening verification is a §11.4 PASS-bluff. The failure mode: source file exists → build packages it → source-layer checks pass → artifact produced with asset stripped or misconfigured, and no gate opens the artifact to verify.

Required per consuming project: one challenge script per artifact type that opens the produced artifact and verifies every declared user-visible asset. The challenge MUST run as part of the standard QA gate. Classification: universal (per §11.4.17). No escape hatch. See Constitution §11.4.38 for the full mandate.

Full-suite retest before release tag (§11.4.40, User mandate 2026-05-17)

Release tag MUST NOT be created until a COMPLETE retest with ALL existing tests runs on a clean baseline AFTER every workable item in the batch is done, fixed, polished, individually verified. Spot-check retests FORBIDDEN — miss interaction defects. Comprises 7 steps: (1) pre-build full sweep, (2) post-build full sweep, (3) on-device 4-phase cycle on EVERY owned device, (4) meta-test full mutation sweep, (5) Challenge bank full sweep, (6) Issues.md/Fixed.md state audit, (7) CONTINUATION.md sync check. Typically 12–48h elapsed, NOT optional, NOT abbreviated. Composes with §11.4.4 (per-fix retest still required at fix granularity) + §11.4.7 (full-suite is authoritative baseline for closures) + §11.4.39 (per-feature on-device validation runs as step 3). Classification: universal (per §11.4.17). No escape hatch. See Constitution §11.4.40 for the full mandate.

Pre-force-push merge-first (§11.4.41, User mandate 2026-05-17)

Forensic anchor — verbatim user mandate (2026-05-17):

> “make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides!”

Any force-push authorised under CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline: (1) fetch every remote (`git fetch --all --prune --tags`); (2) integrate every

divergent commit locally (rebase / merge / operator-confirmed cherry-pick per appropriate strategy); (3) audit the integrated tree (no conflict markers anywhere in governance + source + test files; no file silently dropped; previously-passing tests still pass; captured-evidence artefacts still validate); (4) only THEN execute `git push --force-with-lease`.

Two-gate composition with CONST-043. §11.4.41 does NOT relax CONST-043 — it adds a SECOND mechanical gate. CONST-043 alone authorises a push that loses remote work; §11.4.41 alone risks pushing without operator awareness. Both required.

Three failure modes prevented: remote-side content loss (parallel sessions overwritten), stale-state acts (–force-with-lease reads stale local refs without prior fetch), conflict-driven corruption (markers committed verbatim — observed 2026-05-17 in helix_qa + containers governance files). Verification artefact: `docs/changelogs/<tag>.md` “Force-push merge-first audit” section captures fetch output, divergence log, integration strategy, conflict-marker scan, test delta, push output with lease SHA, + CONST-043 authorisation quote. Gate `CM-FORCE-PUSH-MERGE-FIRST` + paired mutation. Classification: universal (§11.4.17). No escape hatch. Composes with §9.2, §11.4.4, §11.4.6, §11.4.26, §11.4.32, §11.4.37, §11.4.40, CONST-043, CONST-047. See Constitution §11.4.41 for the full mandate. ### Iteration-discipline mandate (§11.4.42, User mandate 2026-05-18)

Work proceeds in priority-ordered cycles. Five mandatory steps per cycle: (1) select TOP + MIDDLE critical only (defer LOW until critical batch closed), (2) batch implementation per §11.4.4 + §11.4.9, (3) smoke gate (<30 min), (4) ONLY if smoke GREEN and no new user/operator report → §11.4.40 full retest (12–48 h), (5) release-ready OR loop back. Composes with §11.4.4 / §11.4.7 / §11.4.9 / §11.4.34 / §11.4.40 — §11.4.42 is the meta-loop conductor binding them. Every PASS in smoke and full retest MUST carry positive captured evidence (§11.4.2 + §11.4.5). Tests AND Challenges bound equally. No escape hatch. Classification: universal (per §11.4.17). See Constitution §11.4.42 for the full mandate.

TDD-Fix-Discipline (§11.4.43, User mandate 2026-05-18)

Every fix follows 5-step workflow: **RED** (failing test FIRST, real product defect, captured evidence) → **LIVE-ADB-PROBE** (try on running device when feasible; INFEASIBLE for kernel / framework / HAL / vendor / init.rc / sepolicy / ro.* / Android.bp — must rebuild; cite `LIVE_PROBE_INFEASIBLE: <reason>` in commit) → **GREEN** (source patch, batched per §11.4.9, four-layer coverage per §11.4.4) → **VERIFY** (re-run RED test, PASSes under SAME

conditions per §11.4.7, 10-iteration reliability per §11.4.42) → **DOCUMENT** (Issues.md → Fixed.md with type-aware closure §11.4.33, CLAUDE.md Applied Fixes row, changelog, guides, HelixQA bank, CONTINUATION.md §12.10 — all in SAME commit). Test-after-fix is a §11.4 PASS-bluff: test agrees with fix but does not catch the bug. No escape hatch. Classification: universal (§11.4.17). See Constitution §11.4.43 for the full mandate.

Document revision header (§11.4.44, User mandate 2026-05-18)

Every IN-scope tracked Markdown document (Issues.md / Issues_Summary.md / Fixed.md / Fixed_Summary.md / CONTINUATION.md / docs/guides/** / docs/research/** / docs/scripts/** / docs/changelogs/** / docs/superpowers/plans/** / docs/hardware/** / every other docs/*.md) carries `**Revision:** N` (monotonic positive integer) + `**Last modified:** YYYY-MM-DDTHH:MM:SSZ` directly below the H1 title. CLAUDE.md / AGENTS.md / README / LICENSE / VERSION / rendered HTML / rendered PDF are OUT of scope. Agents MUST read the `**Revision:** N` literal from the document head when reporting document state, NEVER infer freshness from filename mtime or commit log — both can lie. Agents writing to an IN-scope doc MUST invoke `scripts/doc_revision_bump.sh <file>` as the final step before staging, OR rely on the pre-commit hook. Agents MUST NOT manually edit the revision number — only the bump script is authoritative.

Pre-build gates `CM-DOC-REVISION-HEADER-PRESENT` + `CM-COVENANT-114-44-PROPAGATION` + paired mutations. Composes with §12.10 (CONTINUATION.md header reuse), §11.4.12, §11.4.22, §11.4.23, §11.4.18. No escape hatch. Classification: universal (§11.4.17). See Constitution §11.4.44 for the full mandate.

Integration-status-doc maintenance (§11.4.45, User mandate 2026-05-18)

Every non-trivial domain integration MUST have a `docs/<domain>/<integration>/Status.md` carrying the §11.4.44 revision header + auto-synced HTML+PDF + auto-colorized per §11.4.23 + sync wrapper + captured-evidence table (every claim cites the test log / recording / sink-probe path) + closed status vocabulary (PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED) + operator-blocked items at the top + reference from CONTINUATION.md §3 when non-terminal. §11.4.45 is the generic form of §12.10 applied to every integration domain.

Pre-build gates `CM-COVENANT-114-45-PROPAGATION` + `CM-AF-INTEGRATION-STATUS-DOCS` + 4 paired mutations. Composes with §11.4.5 / §11.4.12 / §11.4.13 / §11.4.15 / §11.4.22 / §11.4.23 / §11.4.44 / §12.10. No escape hatch. Classification: universal (§11.4.17). See Constitution §11.4.45 for the full mandate.

Validate-recent-work-before-post-flash-tests (§11.4.46, User mandate 2026-05-18)

Every post-flash run MUST first run a recent-work validation pass (targeted tests for Issues.md In-progress / Ready-for-testing / Reopened + Fixed.md last-7-days + CONTINUATION.md §3 active work). Full `test_all_fixes.sh` runs ONLY after 100% green.

Helper: `scripts/testing/recent_work_validate.sh --device <serial>` writes `/data/local/tmp/.recent_work_validated`; full suite refuses without it. Each recent fix needs paired §11.4.43 RED-then-GREEN — naked GREEN is a bluff. Composes with §11.4.4 + §11.4.7 + §11.4.40 + §11.4.42 + §11.4.43 + §11.4.44 + §12.10. Pre-build gates `CM-COVENANT-114-46-PROPAGATION` + `CM-AF-RECENT-WORK-VALIDATION-GATE` + `CM-AF-VALIDATION-ARTIFACT-FILE` + paired mutations. Classification: universal (§11.4.17). No escape hatch. See Constitution §11.4.46 for the full mandate.

Firebase data review (§11.4.47, User mandate 2026-05-18)

Before every “bigger working round” (pre-build / pre-flash / pre-tag blocking; daily / post-deployment burn-in non-blocking) the operator/loop MUST execute `scripts/firebase/review_round.sh`. The pass queries Crashlytics (fatals + non-fatals + ANRs) + Analytics + Performance, classifies findings by severity, dedup- maps to existing Issues.md entries via a three-tier algorithm (exact Firebase Issue-ID match → stacktrace-similarity cluster hash → operator merge review), and drafts new Issue entries for unrecognised findings with full Firebase Console URL refs + §11.4.4(a) systematic-debugging output.

Five mandatory elements: 5-trigger cadence, 3-source query (all three sources), Issues.md output with Firebase metadata (Issue IDs + URL + Cluster Hash / KPI / Funnel), 3-tier dedup, and comprehensive root-cause analysis per Issue.

Pre-build gates `CM-COVENANT-114-47-PROPAGATION` + `CM-AF-FIREBASE-REVIEW-CADENCE` + `CM-AF-FIREBASE-ISSUE-XREF` + 3 paired mutations. Composes with §11.4.4 / §11.4.4(a) / §11.4.6 / §11.4.7 / §11.4.10 / §11.4.12 / §11.4.14 / §11.4.15 / §11.4.16 / §11.4.34 / §11.4.42 / §11.4.43 / §11.4.44 / §11.4.45 / §11.4.46. No escape hatch — no `--skip-firebase-review`. Classification: universal (§11.4.17). See Constitution §11.4.47 for the full mandate.

UI-driven video testing (§11.4.48, User mandate 2026-05-18)

Every test that asserts video playback on a secondary display MUST traverse the user-equivalent UI path (launcher icon → app home → content list → tile tap → playback → in-app pause/resume → back button stop). NOT Intent/Broadcast shortcuts (`am start -a VIEW`, `cmd media_session play`). Real `uiautomator dump` element resolution + `input tap X Y`. Coverage: every video-capable app in PRODUCT_PACKAGES + every stream type (progressive HTTP / HLS / DASH / RTMP / file-local / DRM) + every codec (H.264/265/VP9/AV1/MPEG-2/MP4V video; AC-3/E-AC-3/TrueHD/DTS/DTS-HD/MLP/Opus/AAC audio). Secondary display verified via ffprobe-on-captured-mp4 + VOM activeDecoder state. Arvus codec-state cross-check per §11.4.13 + §CG screenshot.

Per §11.4.4 four-layer: pre-build gate `CM-AF-UI-DRIVEN-VIDEO-COVERAGE` + propagation gate `CM-COVENANT-114-48-PROPAGATION` + on-device test framework at `device/rockchip/rk3588/tests/ui_driven/` (Layer 1 helper + Layer 2 per-app drivers + Layer 3 scenarios) + Layer 4 orchestrator `scripts/testing/run_ui_driven_video_suite.sh` + paired meta-test mutations. No escape hatch — no `--use-intent-shortcut`, `--skip-ui-traverse`, `--legacy-intent-mode` flag. Classification: universal (§11.4.17). See Constitution §11.4.48 for the full mandate.

Dual-approach testing (§11.4.49, User mandate 2026-05-18)

Every feature test exercising a user-visible behaviour MUST ship in TWO variants: a UI-driven variant (uiautomator-based, §11.4.48 surfaces A–E) AND an Intent/Broadcast-driven variant (`am start --es` / `am broadcast`-based). Either alone is a §11.4 PASS-bluff for the OPPOSITE half of the stack — UI catches app-side bugs; Intent catches framework/system-server bugs (MediaCodec.configure hook, IVideoOutputManager bind timing, broadcast permission gating).

Shared assertion base `tests/lib/dual_approach_test_base.sh` — both variants share `dat_init` / `dat_start_capture` / `dat_assert_codec_state` / `dat_assert_video_frames` / `dat_assert_audio_channels` / `dat_arvus_dashboard_capture` / `dat_cleanup` / `dat_report_finding`. Both variants gather identical evidence into mirror dirs `qa-results/dual_approach/ <F>/<run-ts>/{ui,intent}/` so orchestrator diffs results and pinpoints which half of the stack contains a bug. Status mismatch between variants is itself a finding.

Kinopoisk 5.1 EAC3 is the canonical first implementation. Both variants are RED per §11.4.43 until the §CN decoder pipeline fix lands. Pre-build gates: `CM-COVENANT-114-49-PROPAGATION` + `CM-AF-`

DUAL-APPROACH-COVERAGE + CM-AF-KINOPOISK-5-1-DUAL- COVERAGE. Three paired meta-test mutations. No escape hatch — no --ui-only / --intent-only / --skip-dual flag. Classification: universal (§11.4.17). See Constitution §11.4.49 for the full mandate.

Deterministic consistency (§11.4.50, User mandate 2026-05-18)

Every test that PASSes MUST have been executed N times (default N=3 normal tests, N=10 cycle-validation suites) against the same firmware MD5 + same device + same topology and produced IDENTICAL PASS in every iteration. A divergent N-iter run is auto-FAIL — there is no "first PASSed therefore X was a flake" path. §11.4.7's intermittent / transient / flake / flaky vocabulary is enforced MECHANICALLY by this mandate, not merely textually.

Coverage: every public API path (Activity / Service / Broadcast receiver / ContentProvider URI / IPC interface / JNI entry / sysprop write / sysfs node / init.rc trigger) MUST have ≥1 dedicated test that drives it. Untested paths surface in a feature-coverage-matrix audit; the threshold ratchets 70 → 85 → 95 → 99 over phases.

Reliability mechanism: ab_run_n_times helper in the project anti-bluff library loops, captures evidence-hash per iter, asserts all N hashes + exit codes identical. NO operator-facing escape converts divergence to PASS.

Pre-build gates: CM-COVENANT-114-50-PROPAGATION + CM-AF-RELIABILITY-CHECK-WIRED + CM-AF-FEATURE-COVERAGE-MATRIX. Three paired meta-test mutations. No escape hatch — no --allow-flake, --first-pass-suffices, --skip-n-iter, --skip-coverage-audit flag. Classification: universal (§11.4.17). See Constitution §11.4.50 for the full mandate.

Live-ADB-First maximization (§11.4.51, User mandate 2026-05-18)

Every fix MUST be classified by rebuild-requirement before commit using the project's per-file-class decision matrix. If LIVE_ADB_TESTABLE (on-device test scripts, host scripts, atmosphere-*.sh boot scripts, persist.* properties, markdown docs, test fixture assets, HelixQA YAML banks), the operator MUST first apply the fix to the running device via adb push / setprop / pm install -r / mount -o remount,rw, run the §11.4.43 RED test live, capture PASS, THEN commit + rebuild + reflash. Commit footer: LIVE_ADB_VALIDATED: yes. If REQUIRES_REBUILD (kernel, framework Java/AIDL, native C++ in APEX, sepolicy, init.rc, ro.* properties,

XML overlays, codec XML in APEX, Android.bp/.mk), the operator proceeds directly to source-side + rebuild. Commit footer: `REQUIRES_REBUILD: <reason>`. Mixed batches use partial.

§11.4.51 REFINES §11.4.43 step 2 with mechanical enforcement. Helper: `classify_fix_rebuild_requirement.sh` walks `git diff --name-only`, looks up each file against the matrix, emits per- file classification + recommended commit-message footer. Unmatched paths classify as `REQUIRES_REBUILD: unmatched-path` (safe default per §11.4.6).

Pre-build gates: `CM-COVENANT-114-51-PROPAGATION` + `CM-AF-CLASSIFY-FIX-HELPER-EXISTS` + `CM-AF-LIVE-ADB-FIRST-COMMIT-MARKER`. Three paired meta-test mutations. No escape hatch — no `--skip-classify` / `--assume-rebuild` / `--no-footer-required` flag.

Classification: universal (§11.4.17). See Constitution §11.4.51 for the full mandate.

Autonomous-Validation (§11.4.52, User mandate 2026-05-18)

Forensic anchor — verbatim user mandate:

> “Make sure we have full automation tests which will do all this work in full automation! IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!”

Every user-facing feature MUST have at least one autonomous validation path: end-to-end via `adb shell` + scripted automation, captured runtime evidence per §11.4.5, PASS/FAIL verdict WITHOUT human presence. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY path is operator-attended is a §11.4.52 violation: the path does not scale to CI, does not run on every commit, does not survive operator unavailability, and produces the exact “tests pass but feature doesn’t work for users” failure mode §11.4 forbids.

Acceptable autonomous paths: instrumentation APK (SDK-API exercises + JSON result file), headless intent dispatch + state poll (`am start --es` + `dumpsys/proc/<pid>/maps` polling), ADB-driven uiautomator (ONLY if hierarchy has ≥1 clickable node — empty hierarchy demands fallback to APK/intent), network-side sink probe (§11.4.13), HelixQA autonomous QA session (§11.4.27).

Coverage ledger (§11.4.25) classifies each feature as `AUTONOMOUS_VERIFIED` / `AUTONOMOUS_DESIGNED` / `OPERATOR_ATTENDED_ONLY` / `NOT_APPLICABLE`. `OPERATOR_ATTENDED_ONLY` blocks release until migrated; cite a tracked work item per §11.4.15 + §11.4.16.

Pre-build gates `CM-COVENANT-114-52-PROPAGATION` + `CM-AF-AUTONOMOUS-PATH-PER-FEATURE`. Paired mutations. No escape hatch — no `--allow-operator-attended-only` / `--skip-autonomous-path` / `--manual-validation-suffices` flag.

Classification: universal (§11.4.17). See Constitution §11.4.52 for the full mandate.

Fixed_Summary parity (§11.4.53, User mandate 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18T17:55Z):

> "Note: Just like for Issues we have Issues_Summary, for Fixed we MUST HAVE Fixed_Summary - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML!"

`docs/Fixed_Summary.md` is the symmetric short-form summary of `docs/Fixed.md`. MUST be regenerated whenever `Fixed.md` changes. HTML + PDF exports MUST travel with the markdown. Stale exports are §11.4.53 violations regardless of whether the underlying `.md` is correct. Same discipline as §11.4.12 Issues_Summary applied to Fixed.md.

Generator: `scripts/testing/generate_fixed_summary.sh` (canonical). Auto-sync wrapper: `scripts/testing/sync_issues_docs.sh` regenerates BOTH Issues_Summary AND Fixed_Summary in one shot, exports HTML + PDF, colorizes per §11.4.23, re-renders PDFs. MUST be invoked after any edit to `Fixed.md`. No `--issues-only` flag exists.

Sort order: closure date DESC (most-recent-Fixed first), §-letter / Fix-# secondary. Documented at top of generated file.

Composes with §11.4.12 (Issues_Summary sibling), §11.4.19 (atomic migration trigger), §11.4.23 (colorizer), §11.4.33 (type-aware closure terminal values), §11.4.44 (revision header), §12.10 (CONTINUATION.md resumption).

Pre-build gates `CM-FIXED-SUMMARY-SYNC` + `CM-COVENANT-114-53-PROPAGATION`. Paired mutations. No escape hatch — no `--skip-fixed-summary-sync`, `--issues-only`, `--summary-not-applicable` flag.

Classification: universal (§11.4.17). See Constitution §11.4.53 for the full mandate.

ATM-NNN ticket identifier (§11.4.54, User mandate 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Every workable item in Issues / Issues_Summary / Fixed / Fixed_Summary MUST carry a stable, unique, auto-incremental ATM-NNN ticket identifier."

Every workable item heading in `docs/Issues.md` AND `docs/Fixed.md` MUST carry `[ATM-NNN]` (form `## §X.Y. [ATM-NNN] <title>`, zero-padded ≥3 digits). Allocated by `scripts/testing/assign_atm_ticket_ids.sh` and persisted to `scripts/testing/.atm_ticket_state.json` (jsonl). Identifiers are monotonic, never renumbered, never reused, no gaps. Issues_Summary.md and Fixed_Summary.md MUST carry an `ATM ID` column as the leftmost data column.

Composes with §11.4.15 + §11.4.16 + §11.4.19 + §11.4.33 + §11.4.12 + §11.4.53 + §11.4.55 + §11.4.57.

Pre-build gates `CM-ATM-TICKET-IDS-COMPLETE` + `CM-ATM-TICKET-IDS-UNIQUE` + `CM-ATM-TICKET-IDS-MONOTONIC` + `CM-COVENANT-114-54-PROPAGATION`. Paired mutations. No escape hatch.

Classification: universal (§11.4.17). See Constitution §11.4.54 for the full mandate.

Reopens-history tracking + per-item Reopens.md (§11.4.55, User mandate 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Add a Reopens-count column. For any item whose reopens-count > 0, create docs/issues/ATM-NNN/Reopens.md (+ HTML + PDF) with comprehensive reopen history + Fixed cycles."

Every workable item with `reopens_count > 0` MUST have `docs/issues/<ATM-NNN>/Reopens.md` (+ HTML + PDF). Required content: §11.4.44 revision header, item identification, cycle counters, chronological timeline (By AI/User per §11.4.34, On ISO date, Event, Reason, Evidence, Outcome), reasoning chain for each closure, most-recent state-change pointer.

Issues_Summary.md and Fixed_Summary.md MUST carry a `Reopens` column; count > 0 hyperlinks to per-item Reopens.md.

Composes with §11.4.34 (per-event capture), §11.4.54 (ATM-NNN path), §11.4.44 (revision header), §11.4.45 (Status.md analogue), §11.4.53 (Fixed_Summary parity — Reopens column symmetric).

Pre-build gates `CM-REOPENS-DOC-EXISTS-WHEN-COUNT-GT-ZERO` + `CM-REOPENS-DOC-REVISION-HEADER` + `CM-REOPENS-COL-IN-SUMMARIES` + `CM-COVENANT-114-55-PROPAGATION`. Paired mutations. No escape hatch.

Classification: universal (§11.4.17). See Constitution §11.4.55 for the full mandate.

Status_Summary parity + two-audience format (§11.4.56, User mandate 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Every Status.md doc gets a Status_Summary parity companion ALWAYS in sync, exported to HTML + PDF. Two-page format: page 1 = non-developer audience (team-specific), page 2 = software engineer summary."

For every `docs/<domain>/<integration>/Status.md` a companion `Status_Summary.md` MUST exist with: §11.4.44 revision header; **Page 1 — For the** (audience-specific — audio team for `docs/dolby/*`, video team for `docs/video/*`, etc.) plain- language summary, What works, What's broken or pending, Operator actions — NO §-letter jargon, NO captured-evidence file paths; **Page 2 — For software engineers** — §-letter refs, gate names, commit hashes, captured-evidence paths, ATM-NNN cross-refs. HTML + PDF exports travel with the markdown.

Generator: `scripts/testing/generate_status_summary.sh <Status.md path>`. Invoked from `sync_integration_status.sh` (§11.4.45 wrapper) on every Status.md update.

Composes with §11.4.45 (Status.md — Status_Summary COMPLEMENTS), §11.4.12 + §11.4.53 (parity discipline), §11.4.44 (revision header), §11.4.23 (colorizer), §12.10 (CONTINUATION.md).

Pre-build gates `CM-STATUS-SUMMARY-EXISTS-FOR-EVERY-STATUS` + `CM-STATUS-SUMMARY-TWO-AUDIENCE` + `CM-STATUS-SUMMARY-REVISION-HEADER` + `CM-COVENANT-114-56-PROPAGATION`. Paired mutations. No escape hatch.

Classification: universal (§11.4.17). See Constitution §11.4.56 for the full mandate.

README.md doc-link section + revision metadata (§11.4.57, User mandate 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

> "Add a doc-link section to README.md — links to Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + ALL Status docs + their exports. Each link shows revision + last-modified."

Every project's top-level `README.md` MUST contain a section titled `Tracked-Items + Status Documents` (heading MUST contain literal `Tracked-Items`). The section is a markdown table with columns: `Document`, `Last modified`, `Revision`, `Markdown`, `HTML`, `PDF`. Links to: Issues.md + Issues_Summary.md, Fixed.md + Fixed_Summary.md, CONTINUATION.md, every `docs/**/Status.md` + its Status_Summary.md pair. Status docs auto-discovered by the generator via `find docs -name 'Status.md'`.

Generator: `scripts/testing/update_readme_doc_links.sh` extracts each doc's revision + last-modified from its §11.4.44 header, renders the table, replaces the section between explicit markers (`<!-- doc-link-section:begin -->` / `<!-- doc-link-section:end -->`). Invoked from `sync_issues_docs.sh` AND `sync_integration_status.sh`.

Composes with §11.4.12 + §11.4.19 + §11.4.53 (Issues / Fixed / summaries), §11.4.44 (revision-header data source), §11.4.45 + §11.4.56 (Status pairs), §12.10 (CONTINUATION.md).

Pre-build gates `CM-README-DOC-LINK-SECTION-PRESENT` + `CM-README-DOC-LINK-ROWS-COMPLETE` + `CM-README-DOC-LINK-FRESHNESS` + `CM-COVENANT-114-57-PROPAGATION`. Paired mutations strip markers, remove rows, backdate `Last modified`, strip anchor literal. No escape hatch.

Classification: universal (§11.4.17). See Constitution §11.4.57 for the full mandate.

Parallel-development methodology (§11.4.58, User mandate 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19T~05:00Z MSK):

> "We MUST CREATE adjusted improoved version of working methodology where multiple workable items could be done in parallel, all come into the central (main) branch as they are done, then we at particular moment rebuild and reflash the System and in background full testing with validation and verification is done. Each parallel work on one workable (or more) item(s) must use parallel agents as much as possible! … Writing in depth tests (all supported types of the

> tests) with Challenges and full HelixQA use is MANDATORY! Every test we execute besides executed with success MUST RESULT in proof that actual functionality being tested REALLY DOES WORK with NO BLUFF of any kind!”

Project work proceeds through the **Parallel Work Unit (PWU) pipeline**. Each PWU is a self-contained workable item with: ATM-NNN identifier per §11.4.54, Issues.md entry per §11.4.15+§11.4.16, file- scope manifest, §11.4.43 RED test, source patch, pre-build gate, post-flash test, paired meta-test mutation per §1.1, HelixQA Challenge bank entry, captured-evidence directory per §11.4.5+§11.4.52.

5-stage pipeline: Stage 1 DEVELOP (parallel PWU agents in isolated worktrees) → Stage 2 MERGE (serial via `commit_all.sh` flock + §11.4.41 4-step merge-first) → Stage 3 REBUILD+FLASH (parallel where hardware allows) → Stage 4 VALIDATE (parallel D3+D4+meta-test+coverage) → Stage 5 SWEEP (parallel HelixQA + Fixed.md migration + README refresh). Stage 1 of round N+1 overlaps with Stages 4-5 of round N — throughput multiplier.

Synchronization: 4-layer lock hierarchy (parent flock / per-submodule git / contention-path advisory locks / per-PWU worktree). Disjoint-scope PWUs fully parallel.

Anti-bluff merge-time enforcement (mandatory, all four): RED-test captured (§11.4.43), paired meta-test mutation (§1.1), 3-iter deterministic-consistency (§11.4.50), captured-evidence per §11.4.5. Metadata-only / configuration-only / absence-of-error PASS REJECTED. HelixQA Challenge MANDATORY for every user-visible PWU.

Pre-build gates `CM-PWU-LOCK-HIERARCHY` + `CM-PWU-ANTI-BLUFF-COVERAGE` + `CM-PWU-MERGE-QUEUE-DISCIPLINE` + `CM-PWU-PARALLEL-AGENT-LIMIT` + `CM-COVENANT-114-58-PROPAGATION`. Paired mutations cover each gate. No escape hatch.

Classification: universal (§11.4.17). See Constitution §11.4.58 for the full mandate. Project-specific implementation reference in consumer-side `docs/guides/PARALLEL_DEVELOPMENT_METHODOLOGY.md`.

README always-sync (§11.4.59, User mandate 2026-05-19)

`README.md` is a §11.4.12-class always-sync document. HTML + PDF exports refresh on every update via `scripts/testing/sync_readme_export.sh` (pandoc + weasyprint); auto-invoked by `sync_issues_docs.sh` so a single doc-sync run refreshes Issues / Issues_Summary / Fixed / Fixed_Summary / CONTINUATION / README (md + html + pdf). README carries a §11.4.44 revision header and a Documentation Map section linking to every Status.md + Status_Summary.md + spec + plan + guide + script-

companion doc + changelog + the constitution submodule, plus per-audience navigation (end user / developer / QA / agent). Pre-build gate `CM-README-EXPORT-SYNC` enforces mtime parity (README.html + README.pdf ≥ README.md). Paired meta-test mutation backdates HTML+PDF → gate FAILs. No escape hatch — no `--skip-readme-sync`, `--no-readme-export`, `--readme-stale-OK` flag. Composes with §11.4.12 + §11.4.18 + §11.4.44 + §11.4.45 + §11.4.53 + §11.4.56 + §11.4.57 + §12.10.

Classification: universal (§11.4.17). See Constitution §11.4.59 for the full mandate.

Documentation always-sync composite covenant (§11.4.60, User mandate 2026-05-19)

Eight doc classes — Issues, Issues_Summary, Fixed, Fixed_Summary, CONTINUATION, README, every Status.md (domain-scoped), every Status_Summary.md — MUST be in sync across `.md` + `.html` + `.pdf`. Composite pre-build gate `CM-DOCS-COMPOSITE-SYNC` walks all 8 classes (Status fleet via recursive `docs/**` find), FAILs the build if ANY `.html` or `.pdf` mtime is older than its `.md`. Per-class anchors §11.4.12 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §12.10 govern individually; the composite catches what they miss when one is silently bypassed. Paired mutation backdates `docs/Issues.html` → gate FAILs. No escape hatch.

Classification: universal (§11.4.17). See Constitution §11.4.60 for the full mandate.

Credentials-handling (§11.4.10)

Forensic anchor — verbatim user mandate (2026-05-12):

> "Credentials or any secret and sensitive data MUST NOT leak!"

`.env` patterns git-ignored project-wide. Runtime-load-only — tests load credentials at execution time from operator-populated files under `scripts/testing/secrets/` (or per-project equivalent). Test scripts MUST NEVER print or log credentials.

Host-session safety (§12)

Project procedures MUST NOT use more than **60%** of total system RAM. Heavy work wrapped in bounded execution scopes so the kernel OOM-kills only the scope — `user@<uid>.service` stays alive. Hibernating / suspending / logging out the user is FORBIDDEN (directly OR indirectly).

Continuation document (§12.10)

`docs/CONTINUATION.md` MUST exist at the project root and reflect the live state of the work. Every non-trivial state change updates it in the same commit. Any agent must be able to resume work exactly where the previous session left off by reading this single file.

Data safety (§9)

Hardlinked backup before any destructive operation. Post-op gate MUST pass. Force-push requires explicit per-session human authorization. Every history rewrite gets a `docs/changelogs/` audit entry.

CLI workflow expectations

1. **Read before write.** Use file-reading tools to inspect the current state before editing. Edits in the dark are how bugs sneak in.
2. **Use the project's commit wrapper.** Direct `git add` / `git commit` is forbidden unless explicitly carved out.
3. **Commit messages cite forensic evidence** for any non-trivial fix (logs, captures, /sys readings, dropbox entries, strace). Speculative narratives are §11.4.6 violations.
4. **Update CONTINUATION.md** in the same commit as the work (§12.10).
5. **Run pre-build / post-build / runtime gates** before declaring a change complete. NEVER report "done" without running the relevant gate.
6. **Cite external research sources** for non-trivial fixes (§11.4.8). Either a citation OR the literal "NO external solution found — original work".
7. **Refuse to bluff.** If a step requires information you don't have, say so and STOP. Don't guess.

Hierarchy of project authority

When a project files declares conflicting rules with this base AGENTS.md, resolve in this order:

1. **Project's `docs/guides/<PROJECT>_CONSTITUTION.md`** — most specific, project-tailored.
2. **`constitution/Constitution.md`** (this submodule) — universal.
3. **Project's root `CLAUDE.md` / `AGENTS.md`** — agent-facing summaries; lossy compared to the Constitution.
4. **This `constitution/CLAUDE.md` / `AGENTS.md`** — universal agent-facing summary; lossy compared to the Constitution.

If a project's own files contradict the Constitution, that is itself a Constitution violation and MUST be reported. Do NOT silently follow the contradicting project file.

When stuck

- Read `constitution/Constitution.md`.
- Cross-reference the project's CLAUDE.md / AGENTS.md.
- If still unclear, ask the operator. Do NOT guess. Do NOT bluff.

---

§11.4.65 — Universal Markdown export mandate (User mandate, 2026-05-19)

Every Markdown document inside the project that is NOT part of an application or service's source-code tree MUST have synchronized `.html` and `.pdf` siblings. Includes: project-root `*.md`, `docs/**/*.md`, `scripts/**/*.md` (doc-format companion docs), owned-submodule top-level README.md / CLAUDE.md / AGENTS.md / CHANGELOG.md and their `docs/**/*.md`, `constitution/**/*.md`, owned HelixQA submodules' equivalents. Excludes: `external/**`, `prebuilts/**`, `packages/modules/**`, `kernel-5.10/**`, `out/**`, `build/**`, application/service source-code trees, and third-party submodules NOT in the owned set. Every edit triggers regeneration via `scripts/testing/sync_all_markdown_exports.sh` (pandoc HTML + weasyprint PDF, `timeout 60` per file, capped at 500 candidates). HTML + PDF mtime MUST be ≥ source `.md` mtime at all times.

Pre-build gates `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` + `CM-COVENANT-114-65-PROPAGATION`. Paired meta-test mutations. Composes with §11.4.12 / §11.4.18 / §11.4.23 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.59 / §11.4.60 / §11.4.63 / §11.4.64. No escape hatch — no `--skip-md-exports`, `--no-pdf-only`, `--md-export-not-applicable` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.65.

Non-compliance is a release blocker regardless of context.

§11.4.66 — Blocker-resolution interactive-clarification mandate (User mandate, 2026-05-19)

When any task is blocked (operator decision, hardware access, external authorization, ambiguous scope), the agent MUST: (1) research what's doable from the agent side without operator input; (2) calculate minimum-viable operator input; (3) construct 2–4 mutually-exclusive options with one marked "Recommended" and each stating what the agent does after that answer; (4) present via the platform's interactive question mechanism (`AskUserQuestion` on Claude Code) — NEVER free-text "what would

you like?” for closed- set decisions; (5) after the answer, resume work without follow-up round-trips. Composes with §11.4.6 / §11.4.7 / §11.4.40 / §11.4.41 / §11.4.42 / §11.4.52. No silent waiting; no bulk-text questions when interactive options would do.

Pre-build gate `CM-COVENANT-114-66-PROPAGATION` enforces the anchor literal across the 42-file consumer fleet. Paired meta- test mutation strips the literal → gate FAILs. No escape hatch — no `--skip-ask`, `--silent-wait`, `--free-form-only` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.66.

Non-compliance is a release blocker regardless of context.

§11.4.67 — Shell-script target-shell-parseability mandate (User mandate, 2026-05-19)

Forensic anchor — direct user mandate (verbatim, 2026-05-19): “any issue we spot must be fixed, bash scripts as well if they are broken!” + “Make sure that this is mandatory rule!”

Every shell script that may be invoked under a target shell other than the one in its shebang MUST parse cleanly under that target shell. Forensic incident: `device/rockchip/rk3588/tests/test_all_fixes.sh:114` used bash-only `exec > >(tee -a "$f") 2>&1` on a `sh script.sh` callsite — Android mksh parses the whole script BEFORE executing, so the runtime `[ -n "${BASH_VERSION:-}" ]` guard could not save it. Fixed by wrapping in `eval 'exec > >(tee …) 2>&1'` so the parser sees only a string.

Closed-set scope: every tracked `.sh` under `device/rockchip/rk3588/tests/`, `scripts/`, `scripts/testing/` (and equivalent paths in owned submodules). OUT of scope: `external/`, `prebuilts/`, `packages/modules/`, `kernel-5.10/`, `out/`, `build/`, `scripts/legacy/`. Mandatory invariants: (1) every in-scope script parses under `sh -n`; (2) bash-only constructs (`>(...)`, `<(...)`, `[[ ]]`, `<<<`, arrays, `${var^^}`, etc.) MUST be wrapped in `eval` OR guarded by bash-only loading; (3) shebangs honest — `#!/bin/bash` only if bash actually expected; (4) fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51.

Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` runs `sh -n` on every in-scope script. Propagation gate `CM-COVENANT-114-67-PROPAGATION` enforces the anchor literal across the 44-file consumer fleet. Paired mutations: inject bash-only outside `eval` → parse gate FAILs; strip `11.4.67` literal → propagation gate FAILs. No escape hatch — no `--skip-parseability-check`, `--bash-only-script`, `--runtime-guard-suffices` flag.

Canonical authority: constitution submodule `Constitution.md` §11.4.67.

Non-compliance is a release blocker regardless of context.

§11.4.68 — Positive sink-side / downstream evidence mandate (User mandate, 2026-05-20)

A test asserting audio or video routing PASS MUST capture and verify positive sink-side or downstream evidence — never config-only, never metadata-only, never PCM-open-state-only. Closed enumeration of acceptable evidence: (1) sink-side codec-state showing non-empty codec matching the expected regex during playback; (2) PCM `hw_ptr` delta strictly positive across the playback window; (3) ALSA ELD demonstrating negotiated channel count + format; (4) ffprobe non-zero frames matching expected codec for video; (5) recording-analyzer matched event per §11.4.2 / §11.4.5 timeline; (6) tinycap RMS amplitude above floor.

Failure modes specifically forbidden: silent SKIP on sink unreachable, empty codec-state treated as evidence, policy-side device field used as proof PCM opened, config-XML-only PASS. All produce the §11.4 PASS-bluff failure mode anchored at D3 forensic 2026-05-20.

Mandatory protections: sink-side libraries expose `*_require_reachable` returning exit code 2 (OPERATOR-BLOCKED); harness propagates 2; audio/video-routing tests capture ≥1 positive evidence per enumeration; anti-stickiness post-stop re-probe. No escape hatch.

Composes with §11.4.2 / §11.4.5 / §11.4.13 / §11.4.14 / §11.4.46 / §11.4.49 / §11.4.50 / §11.4.52. Pre-build gates `CM-COVENANT-114-68-PROPAGATION` + `CM-POSITIVE-SINK-EVIDENCE-PER-AUDIO-TEST` + paired meta-test mutations per §1.1.

Canonical authority: constitution submodule `Constitution.md` §11.4.68.

Non-compliance is a release blocker regardless of context.

§11.4.69 — Universal sink-side positive-evidence taxonomy + mechanical enforcement (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> “THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! Not just for audio but for every single piece of the System!!! Proper full automation when executed with success MUST MEAN that manual testing will be as much positive at least regarding the success results! … Solution

> MUST BE universal, generic that solves working flows for all System components and for all future and all existing projects! … Everything we do MUST BE validated and verified with rock-solid proofs and anti-bluff policy enforcement and fulfillment!”

Universal generalisation of §11.4.68 (audio-specific) across every user-visible feature class. Closes the PASS-bluff pattern where tests reported green while end users hit broken features (2026-05-19→20 D3 audio “82/84 PASS” + empty Arvus Codec-In-Use).

The mandate. Every user-visible feature MUST map to one entry in the closed-set §11.4.69 sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a2dp, bluetooth_pair, touch_input, sensor, gpu_render, storage_read, storage_write, mediacodec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render — open to additions). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape.

Helper contracts (additive during grace; mandatory after 2026-06-19):

- `ab_pass_with_evidence <description> <evidence_path>` — the new canonical PASS helper. Verifies path exists AND non-empty; emits `PASS: <description> [evidence: <path>]`.
- `ab_skip_with_reason <description> <closed-set-reason>` — reasons: `geo_restricted`, `operator_attended`, `hardware_not_present`, `topology_unsupported`, `network_unreachable_external`, `feature_disabled_by_config`. Forbids `network_unreachable_external` for any taxonomy feature with a sink-side probe.
- Bare `ab_pass` deprecated — WARN pre-grace, FAIL post-grace (2026-06-19).

Mechanical enforcement. Three pre-build gates + three paired §1.1 meta-test mutations:

- `CM-SINK-EVIDENCE-PER-FEATURE` — walks tests for `# §11.4.69 FEATURE: <class>` annotation + verifies taxonomy probe + `ab_pass_with_evidence` use.
- `CM-NO-FAIL-OPEN-SKIP` — audits sink-side probe helpers; FAILs if any code path converts empty/unreachable response to PASS-counting SKIP for a feature class with a sink-side probe.
- `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE` — pre-grace WARN, post-grace FAIL on bare `ab_pass` calls.

Composes with §11.4.1 (FAIL-bluffs forbidden), §11.4.2 (recorded-evidence), §11.4.5 (audio + video 5-layer quality), §11.4.6 (no-guessing), §11.4.13 (sink-side captured-evidence), §11.4.27 (no-fakes-beyond-unit), §11.4.50 (deterministic consistency), §11.4.52 (autonomous-validation), §11.4.68 (audio-specific sink-side — §11.4.69 is the universal generalisation).

No escape hatch — no `--skip-evidence`, `--config-only-pass`, `--allow-fail-open-skip`, `--legacy-ab-pass-permitted` flag. The discipline exists because the 2026-05-20 forensic incident demonstrated the failure: tests reported audio-routing PASS while the user heard nothing and the Arvus Codec-In-Use field was empty.

Propagation gate `CM-COVENANT-114-69-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.69.

Non-compliance is a release blocker regardless of context.

§11.4.70 — Subagent-driven execution is the default (User mandate, 2026-05-20)

Forensic anchor — direct user mandate (verbatim, 2026-05-20):

> "Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!"

When executing implementation plans authored via `superpowers:writing-plans` (or any equivalent task-decomposed execution flow), the **default execution model is subagent-driven** per `superpowers:subagent-driven-development`. Inline execution via `superpowers:executing-plans` is permitted ONLY when (a) the task is trivial AND fits in a single sub-300-line edit, OR (b) the operator explicitly requests inline execution at brainstorm-handoff time.

Subagents bring an isolated context window per task (no conductor context bloat), a structurally separated review seam (conductor reviews subagent output, eliminating self-review blind spots), parallel-PWU compatibility (§11.4.58 — subagents ARE the parallel work units), and resumability across operator absence (subagents resume from on-disk plan + spec inputs).

Composes with §11.4.4 (four-layer coverage), §11.4.6 (no-guessing — subagent's captured output IS the evidence), §11.4.42 (iteration discipline), §11.4.43 (TDD-fix), §11.4.50 (deterministic consistency), §11.4.51 (LIVE_ADB_FIRST), §11.4.58 (parallel-development PWU).

No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff. Pre-build gate `CM-COVENANT-114-70-SUBAGENT-DEFAULT-PROPAGATION` enforces this anchor literal across the ~44-file consumer fleet. Paired meta-test mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md` §11.4.70.

Non-compliance is a release blocker regardless of context.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (User mandate, 2026-05-20)

Everyday-push variant of §11.4.41 (force-push merge-first). Before pushing to any upstream for any repository (main repo or submodule), the agent MUST follow the 5-step pre-push cycle: (1) `git fetch --all --prune --tags`, (2) `git pull --no-rebase <remote> <branch>` for each remote whose tip differs from local, (3) investigate the diff vs OUR previous HEAD by reading every foreign commit's body (what changed, why, how it affects OUR project), (4) integrate mandatory changes with full §11.4.4(b) four-layer anti-bluff test coverage producing REAL captured-evidence proofs, (5) THEN push to every configured remote in cascade order.

Composes with §11.4.41 (force-push case) + §11.4.26 / §11.4.32 / §11.4.37 / §11.4.40 / §11.4.42 / §11.4.43 / §11.4.4(b) / §11.4.5 / §11.4.6.

Applies to parent repo + constitution submodule + every owned submodule + every nested submodule + every HelixQA dependency. Audit-trail per push reconstructable from `docs/changelogs/<tag>.md` + per-repo `git log` evidence.

No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag. Pre-build gate `CM-COVENANT-114-71-PROPAGATION` + paired mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.71.

Non-compliance is a release blocker regardless of context.

§11.4.72 — Audio Top-Priority Mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): “Make sure all fixes for audio are always top priority in main working stream!” The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Audio scope includes §EU HDMI rejection, §EM multichannel HDMI, §ET Arvus integration, §EV/§EW/§EX D3 audio defects, HiFi (Fix #74, #76), AC3 (Fix #106), ES8388 (Fix #103/§104/#105), multichannel LPCM (Fix #112), and every future audio-stack improvement.

Composes with §11.4.42 (iteration-discipline — audio sits at apex of priority order) + §11.4.58 (parallel-development PWU — background research subagents run concurrently but do NOT preempt audio on the main-stream serial dispatch queue).

Operative rule: any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. No escape hatch — no “but this non-audio task is faster” or “but this research is more interesting” override. Audio-stack regressions are user-perceptible and high-impact (D3 silent post-flash is a release blocker), while research and refactors can wait.

Canonical authority: constitution submodule `Constitution.md` §11.4.72.

Non-compliance is a process violation regardless of context.

§11.4.73 — Main-specification document versioning + revision discipline (User mandate, 2026-05-20)

Direct user mandate (verbatim): “Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Document MUST BE updated ALWAYS to follow the versioning rules we are applying here + revision and other properties we have!”

Applies only when a project recognises a main specification document. Two-axis versioning: **primary** (V1/V2/V3/…) bumps for major rewrites (old primary archived); **secondary** (`Revision`) bumps for additive operator requirements (matches the §11.4.61 metadata-table `Revision` integer). Every operator-mandated requirement MUST land in the spec as part of the work that implements it. Cross-doc propagation copies MUST reference the active spec file, not a stale archived version. Composes with §11.4.44, §11.4.61, §11.4.59, §11.4.65.

Canonical authority: constitution submodule `Constitution.md` §11.4.73.

Non-compliance is a release blocker.

§11.4.74 — Submodule-catalogue-first discovery + extend-don't-reimplement (User mandate, 2026-05-20)

Direct user mandate (verbatim): "We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in `vasic-digital` and `HelixDevelopment` organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times. For any missing features we MUST IMPLEMENT them properly and extend those Submodules further!"

Before scaffolding any new module / package / helper / utility, the agent MUST: (1) survey `vasic-digital` + `HelixDevelopment` orgs on GitHub + GitLab; (2) reuse an existing Submodule when it covers ≥ 80% of the functionality; (3) extend in-place via upstream PR when 80%+ matches but features are missing — never duplicate; (4) document the survey result in the relevant tracker row with `Catalogue-Check: reuse|extend|no-match <org/repo>@<sha>`.

Every Submodule in the catalogue is subject to the same development-cycle rules (§11.4 anti-bluff, §1.1 paired mutations, §11.4.10 credentials, §11.4.61 metadata + ToC, §11.4.65 universal export, §11.4.73 spec versioning, §2.1 multi-mirror push, §3 propagation order).

Canonical authority: constitution submodule `Constitution.md` §11.4.74.

Non-compliance is a process violation; severe cases (duplicate implementation landed without catalogue check) are release blockers.

§11.4.75 — Mechanical Enforcement Without Exception (User mandate, 2026-05-20)

Direct user mandate (verbatim): "Why do these violations still happen!? This is a serious problem! We cannot rely on stability nor consistency if we cannot respect our Constitution, mandatory rules and constraints! Is there a way to make this always respected, followed and applied without exception fully and unconditionally!? WE MUST HAVE THIS WORKING FLAWLESSLY!!! Do investigate the root causes of such problems! Once all problems are identified WE MUST apply proper mechanisms for this not to happen NEVER EVER AGAIN!"

The §11.4 covenant historically relied on agent + operator vigilance. Three forensic incidents in 2026-05-19→20 (two stalled remediation subagents + post-§11.4.66 propagation drift requiring catch-up commit `f93b25a92eb`) demonstrated that late-binding enforcement at `pre_build_verification.sh` time fires hours-to-days AFTER the violator commit has already reached every remote. §11.4.75 closes the gap with FIVE independent mechanical enforcement layers — bypassing any single layer does not bypass the discipline:

1. **Local `pre-commit` git hook** — refuses staged `.md` lacking sibling `.html`+`.pdf` (and other staged-only invariants).
2. **`commit_all.sh` integration** — the canonical project commit script invokes the same checks + auto-runs `sync_all_markdown_exports.sh` to self-repair before commit (`_constitution_sibling_check` function).
3. **Local `pre-push` git hook** — re-runs siblings + propagation-gate subset on every commit in the push.
4. **`post-commit` auto-repair hook** — detects orphan `.md` in just-committed manifest, auto-generates siblings via pandoc + weasyprint, creates a `chore(§11.4.75): auto-export ...` follow-up commit. Idempotent + recursion-guarded.
5. **CI/CD workflow** (`.github/workflows/constitution-compliance.yml`) — final defence at remote level. Runs propagation-gate subset on every push to main + every PR; auto-fails on violations.

Helper contracts (mandatory): `scripts/install_git_hooks.sh` (idempotent installer wired into `scripts/setup.sh`), `scripts/git_hooks/{pre-commit,pre-push,post-commit,commit-msg}`, `_constitution_sibling_check` in `scripts/commit_all.sh`. The `commit-msg` hook enforces a `Bypass-rationale: <reason>` footer when `--no-verify` is detected (touch-file marker `.git/ATMO_LAST_BYPASS_ATTEMPT`); `docs/audit/bypass_events.md` accumulates the audit trail per §11.4 captured-evidence requirement.

Five pre-build gates with paired §1.1 meta-test mutations: `CM-COVENANT-114-75-PROPAGATION` (anchor literal across canonical files), `CM-GIT-HOOKS-INSTALL-SCRIPT` (installer present + executable), `CM-GIT-HOOKS-SOURCE-DIR` (4 hook bodies present + executable), `CM-COMMIT-ALL-SIBLING-CHECK` (`_constitution_sibling_check` in `commit_all.sh`), `CM-CI-WORKFLOW-PRESENT` (CI workflow + ≥3 required jobs).

Composes with §1.1 (paired meta-test mutations), §9 (data safety), §11.4 (end-user-quality covenant — Layer 5 CI makes the §11.4 promise mechanical), §11.4.41 (this anchor's renumber from §11.4.74 → §11.4.75 was itself driven by §11.4.41 merge-first discipline), §11.4.65 (universal Markdown export — Layer 1+3+4 are §11.4.65's mechanical seam), §11.4.66 (interactive clarification), §11.4.67 (target-shell-parseability — hooks parse under bash AND mksh), §11.4.71 (pre-push fetch + integrate —

Layer 3 + 5 honour the merge-first pipeline), §11.4.72 (audio top-priority — hooks hold no lock themselves, so do not race against in-flight audio commits), §11.4.73 (main-spec versioning — when spec exists, hooks include it), §11.4.74 (submodule-catalogue-first — hooks can be added to the canonical catalogue for cross-project reuse).

No escape hatch — no `--skip-hooks`, `--bypass-enforcement`, `--allow-orphan-md`, `--ci-not-applicable`, `--mechanical-enforcement-not-needed` flag exists. The `--no-verify` route IS the deliberate audit-trail bypass; §11.4.75 makes the audit trail mechanical via the `commit-msg` footer requirement.

Canonical authority: constitution submodule `Constitution.md` §11.4.75.

Non-compliance is a release blocker regardless of context.

§11.4.76 — Containers-submodule mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): “For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemy / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: `https://github.com/vasic-digital/containers` (`git@github.com:vasic-digital/containers.git`). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!”

§-slot history note. Originally drafted as §11.4.75, renumbered to §11.4.76 per §11.4.71 fetch-before-push after the concurrent `0a70083` landing of §11.4.75 (Mechanical Enforcement).

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), the agent MUST: (1) install `vasic-digital/containers` (`digital.vasic.containers`) as a Git submodule when scaffolding the consuming project; (2) consume via `replace` directive during development + pinned commit SHAs in production; (3) boot infra on-demand via the Submodule’s `pkg/boot` + `pkg/compose` + `pkg/health` APIs so the operator is never required to start `podman machine` / `docker compose up` manually — the boot is part of the test entry point (**on-demand-infra invariant**); (4) extend the Submodule via upstream PR when a runtime / lifecycle primitive is missing — never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components MUST actually boot them via the Submodule. Short-circuit fakes that bypass boot are a §11.4 violation. A passing test MUST imply the infra was up.

Tracker rows touching containerization MUST record `Catalogue-Check: extend vasic-digital/containers@<sha>` (or `reuse`); `no-match` requires demonstrating the Submodule cannot model the workload.

Planned anti-bluff gate `CM-CONTAINERS-USED` scans container-touching PRs for `digital.vasic.containers/...` imports. Paired mutation strips the import + asserts FAIL.

Composes with §11.4.74 (catalogue-first), §11.4.75 (mechanical enforcement), §3 (propagation), §11.4.31 (Submodule-Dependency-Manifest), §11.4.36 (`install_upstreams` on clone), §11.4.28 (Submodules-As-Equal-Codebase), §1.1 (paired-mutation gates).

Canonical authority: constitution submodule `Constitution.md` §11.4.76.

Non-compliance: reinventing compose orchestration in-project is a release blocker.

§11.4.77 — Regeneration-mechanism-required mandate (User mandate, 2026-05-20)

Direct user mandate (verbatim): “We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content!”

Forensic anchor. 2026-05-20T15:00Z: ATMOSphere parent’s audio Tier 1 `commit_all.sh` stalled 4 h on `git add -A` scanning 274 GiB `.git-backup-*` + 159 GiB `RKTools/linux/` + 167 GiB `qa-results/` — all untracked but un-gitignored. Bare `.gitignore` fix would orphan every fresh clone (missing RKTools, missing test infra, missing build outputs).

Every `.gitignore` entry excluding (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project MUST carry a documented + automated mechanism to either **re-obtain** (vendor tarball, SDK installer, package registry, git submodule, object store) OR **re-generate** (build pipeline, code-gen, asset render, captured-evidence replay, container build).

Required artefacts per qualifying `.gitignore` entry: (1) `.gitignore-meta/<entry-slug>.yaml` declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source + integrity hash + requires-network + requires-credentials; (2) post-clone bootstrap entry (`scripts/setup.sh` or canonical equivalent) running the mechanism non-interactively; (3) pre-build gate verifying regenerated content present OR stamp `.gitignore-`

`meta/.regenerated/<slug>.ok` recent; (4) README + `docs/guides/*.md` describing the mechanism + manual fallback + time/disk budget + per-§11.4.10 credentials.

No escape hatch: bare `.gitignore` additions without the mechanism are §11.4 PASS-bluff variants — codebase appears complete but fresh clone cannot build / run. No `--skip-regen-mechanism`, `--gitignore-is-enough`, `--operator-already-has-content` flag.

Planned anti-bluff gate `CM-GITIGNORE-REGEN-MECHANISM` scans every `.gitignore` addition for matching `.gitignore-meta/` sibling. Paired §1.1 mutation strips a required YAML key → gate FAILs.

Composes with §11.4.6 (no-guessing — verify mechanism on sandbox), §11.4.65 (HTML/PDF siblings are a `re-generate` instance), §11.4.66 (interactive clarification if mechanism unknown), §11.4.71 (re-validate vendor integrity before push), §11.4.74 (extend a reusable downloader Submodule), §11.4.75 (pre-commit + CI replay refuse bare additions), §11.4.76 (container images regenerated via `vasic-digital/containers`), §9 / §9.2 (pre-test mechanism in sandbox), §3 (consuming submodules inherit).

Canonical authority: constitution submodule `Constitution.md` §11.4.77.

Non-compliance is a release blocker regardless of context.